Data Preprocessing for an AI SMS-Based Tutor.

Yusuf Olaniyi (yolaniyi)
Carnegie Mellon University (Africa Campus)
yolaniyi@andrew.cmu.edu

I. INTRODUCTION

Many Rwandan upper-primary pupils have a smartphone or internet. The SMS tutor is aimed at sending a question from the Rwanda Education Board (REB) books [1], receiving a short reply, and judging it based on the official answer. The planned model is a binary classifier with target Y being 0 or 1 based on the correctness of the response it receives. Phase 1 set out the strategy; this report records what happened when it met the data.

The report follows the order of the pipeline. Section II describes how the data changed from plan to execution. Sections III–VII cover missingness, outliers, collinearity, transformation, and dimensionality, each tied to one figure. Section VIII describes the leakage controls, Section IX verifies the output matrices, and Section X gives the strategy-versus-reality decision log (Table IV). All numbers come from the report stats script (`src/report_stats.py`), and all figures can be regenerated with the figure script (`src/figures.py`).

II. DATA AND SCOPE CHANGES

A. Extracting the usable items in the extracted record

In phase 1, an extraction script was used to extract questions and answers from Upper Primary 4 to 6 Teachers Guide and Pupil Material from the REB Books. This produced a raw table of 965×28 (517 open, 176 multiple-choice, 170 fill-in, 91 true/false, and 11 matching items). Phase 1 was planned to keep all five types. However, in Phase 2, I narrowed the scope to multiple-choice (MCQ) and true/false (TF) items, kept the question and the answer in separate tables, and joined them. Open and fill-in items need free-text judgment that the current features cannot support, and the answers to some of them sit inside the question text.

A rule-based cleaning step was used to fix extraction errors and logged each change. It removed 152 page-number fragments, converted 44 items with inline options such as "(a) X (b) Y" to MCQ, split 26 bundled true/false statements into one row each, and removed leaked instructions (10) and "true or false" wording (24). The cleaned table has 359×37 cells. Answer keys were validated: 22 were normalised, and 21 untrustworthy keys (for example, a letter that was not among the options) were set to missing and flagged instead of deleted. Of the 359 items, 170 remained usable (Table I).

TABLE I
ROWS EXCLUDED FROM SUPERVISED USE

Reason	Items
No answer key (guide missing or unmatched)	150
Question text missing	17
Needs a picture or a computer	12
Answer key judged untrustworthy	10
Usable (MCQ 107, TF 63; P4 58, P6 112)	**170**

The merge was checked, not assumed. The 170 questions and 170 answers matched one-to-one on the question unique identifier; the class and unit in each ID agreed with the columns, every TF key was True or False, and every MCQ key was a letter among the options with matching answer text.

B. Deviation: how the target was built

Phase 1 planned a language model to write four replies per item (one correct, three wrong) and a real-life test set. No such replies exist yet. To obtain a real target without inventing data, each MCQ option and each True/False value became one candidate row, and the target (is_correct) comes from the answer key. This gives 520×22 candidate rows (394 MCQ, 126 TF), against about 2,152 planned. The positive rate is 32.6% in the training split (overall splits: MCQ 27.2%, TF 50.0%), not the planned 25%. As a result, the model can now learn to pick the correct option, not to grade a pupil's wording. The language-model replies will be added as further candidate rows without changing the pipeline.

III. MISSINGNESS MECHANISM

Figure 1 shows the pattern for the cleaned table. In the raw table, 29.7% of `answer_key` was missing; in the cleaned MCQ/TF table, it is 48% because the answers in some of the teachers' guides were unavailable, and I set the 21 bad keys to missing. `stem` and `sms_text` are missing for 17 items (4.7%), and `unit_title` for 138 (38.4%).

Missingness of the key depends strongly on class: P4 44.4%, P5 100%, P6 13.6%. Cramér's V between key-missingness and `class_level` was 0.703 (χ^2 = 177.5, df = 2, $p < 0.001$), against 0.197 for `section` (χ^2 = 13.9, df = 5, p = 0.016) and 0.046 for `item_type` (p = 0.38). Missing question text also depended on class (V = 0.265, $p < 0.001$) but not on item type (V = 0.070, p = 0.19). These showed that missingness was random based on the source book, and not completely at random.

Phase 1 had already set aside Little's MCAR test for this table, and the implementation confirmed why. The only numeric columns with gaps (`n_words_question`, `sms_chars`, `sms_segments`) are missing together in the same 17 rows, so there are just two missingness patterns and no pattern with partially observed variables. The statistic has zero degrees of freedom and is undefined. The Cramér's V and χ^2 tests above carry the argument instead.

Answer keys were not imputed, since an invented key becomes a wrong label. Rows without a usable key are excluded from the training data but kept in the cleaned table with `key_missing` and `stem_missing` indicators. A zero length on a row with no question text was recoded to missing, not 0. The candidate table has 0 missing cells among its 15 features, so the median and mode imputers inside the pipeline currently fill nothing. They are kept as a guard for new books. The Phase 1 choice of median over KNN is unchanged.

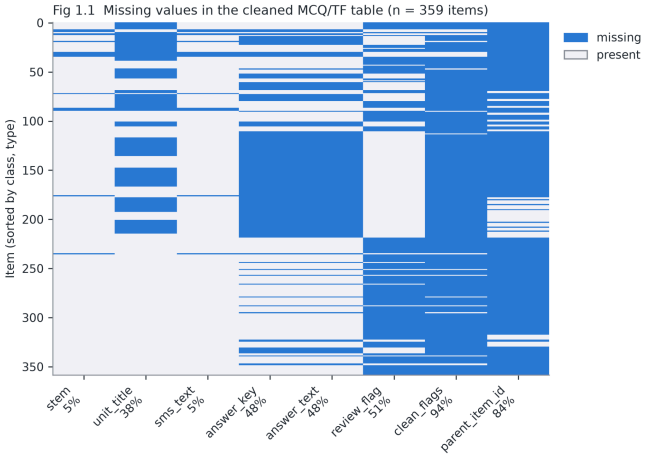


Fig. 1. Missingness pattern of the cleaned MCQ/TF table (n = 359). Blue cells are missing; rows are sorted by class and item type. The key columns are missing in whole blocks (P5), which indicates a source-driven mechanism.

A. Target balance and a shortcut risk

Figure 2 (left) shows 107 correct against 287 incorrect MCQ candidates and 63 against 63 for TF, an overall imbalance ratio of 2.1:1. Stratifying the split by item type keeps the positive rate near 33% in all three partitions (32.6%, 33.3%, 32.5%). Figure 2 (right) shows a problem Phase 1 did not anticipate: the correct MCQ option is b in 57 of 107 items (53.3%), c in 31, a in 15, and d in 4. A uniform test rejects equal positions (χ^2 = 59.4, df = 3, $p < 0.001$). Candidate position is therefore a shortcut that the classifier can exploit without reading the question. I kept it as a feature in this phase so the effect is visible (Section V), but the modelling phase must report results with position features removed.

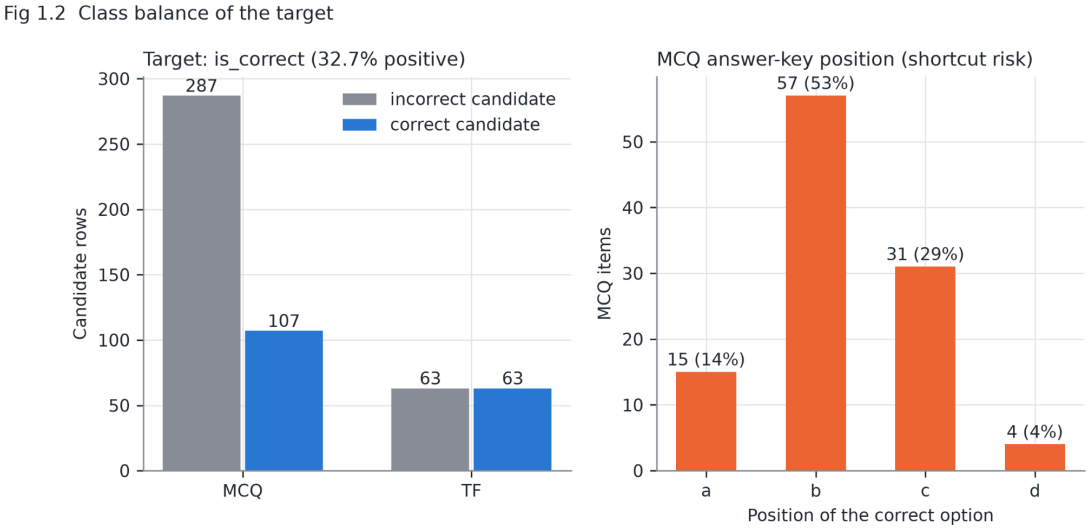


Fig. 2. Left: candidate rows by item type and label. Right: position of the correct option in the 107 usable MCQ items.

IV. OUTLIER TREATMENT AND DISTRIBUTION SHAPES

Fences were learned on the training split (n = 362 candidate rows). Table II lists skewness, the IQR bounds, and what was capped. Phase 1 expected all counts to be strongly skewed (1.8 to 13.5) because it used the full 965-row table with reading passages and multi-part items. After scoping to short MCQ/TF items, the picture is milder: only `sms_segments` (2.32), `cand_overlap` (2.25), `cand_n_words` (1.55), and `cand_len_ratio` (1.17) exceed a skewness of 1.

Two rules were kept from Phase 1. Winsorizing caps values and keeps every row, and when the IQR is zero, the 99th percentile is used instead of a fence that would flag every non-zero value (like in `sms_segments`, `cand_overlap`). One rule was added after I inspected Figure 3. A plain IQR fence on `n_options` (Q1 = 3, Q3 = 4) gave bounds of 1.5 to 5.5 and would have capped 88 rows (24.3%), all of them TF rows whose option count is 0 by design. Raising those zeros to 1.5 would corrupt a structural value, so `n_options` and `cand_position`, which are bounded by the item design, were exempted. In total, 47 cells (1.6% of the 2,896 numeric cells) were capped: 36 in `cand_len_ratio`, 9 in `cand_n_words`, and 2 in `n_words_question`. Capping brought the skewness of `cand_len_ratio` from 1.17 to 0.21 and of `cand_n_words` from 1.55 to 1.05, and no row was deleted.

TABLE II
OUTLIER STATISTICS ON THE TRAINING SPLIT (N = 362). SKEW0: RAW; SKEW1: AFTER CAPPING AND THE LEARNED LOG RULE. |Z| > 3 SHOWN FOR COMPARISON ONLY

Feature	Skew0	Excess kurt.	Q1 / Q3	Upper bound	Capped (%)	Rule	|Z|>3	Skew1
`n_words_question`	0.24	0.04	7 / 11	17	2 (0.6)	IQR fence	2	0.14
`sms_chars`	0.74	−0.05	75 / 137	230	0 (0.0)	IQR fence	0	0.74
`sms_segments`	2.32	3.37	1 / 1	2	0 (0.0)	99th pct.	0	2.32
`n_options`	−1.04	−0.75	3 / 4	—	exempt	bounded	0	−1.04
`cand_position`	0.44	−1.00	0 / 2	—	exempt	bounded	0	0.44
`cand_n_words`	1.55	2.22	1 / 3	6	9 (2.5)	IQR fence	7	0.73
`cand_len_ratio`	1.17	2.50	0.83 / 1.07	1.44	36 (9.9)	IQR fence	7	0.21
`cand_overlap`	2.25	3.74	0 / 0	0.143	0 (0.0)	99th pct.	13	2.22

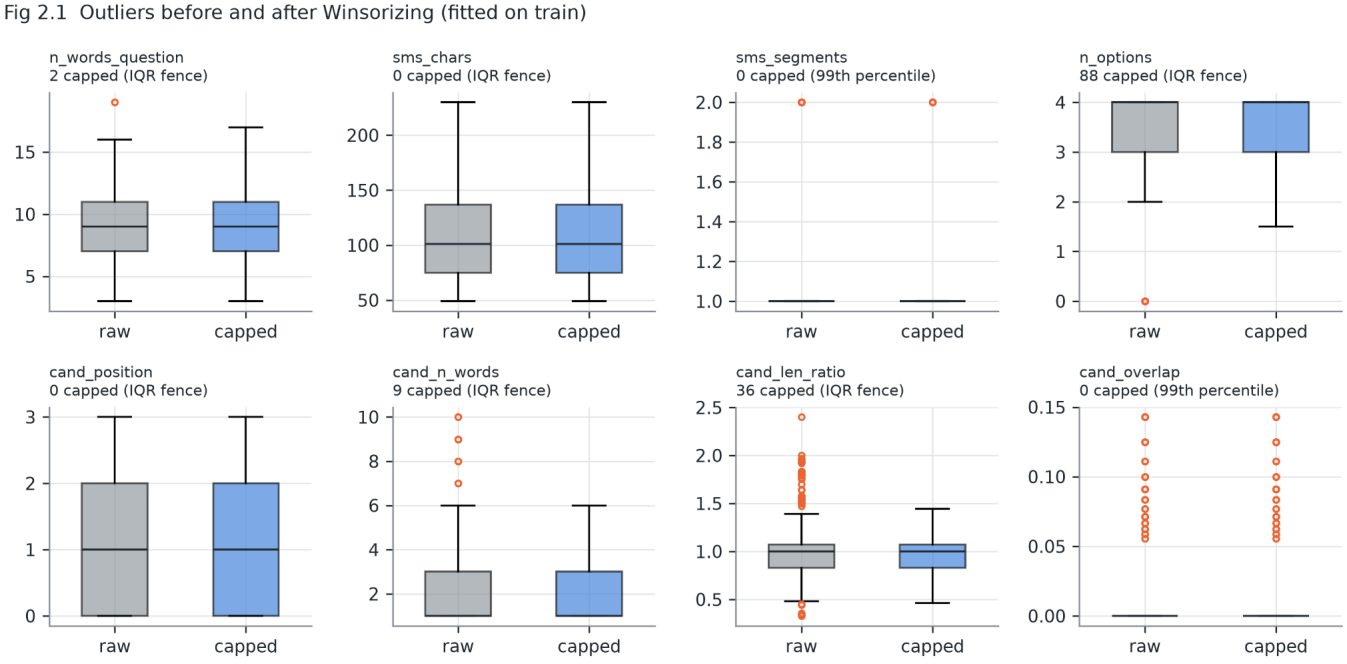


Fig. 3. Raw versus capped distributions of the eight numeric features (training split). Orange circles are points beyond the whiskers; the panel title gives the number capped and the rule used.

V. MULTICOLLINEARITY AND FEATURE SCREENING

Pairwise screen. Figure 4 shows Spearman correlations on the training split. I used rank correlation from the start because the counts are skewed and tied; Pearson reached 0.70 for `sms_chars`–`sms_segments` against a Spearman value of 0.57, so the two measures disagree where the data are two-valued. No pair reached the Phase 1 threshold of $|r| \geq 0.85$. The highest were `sms_chars`–`cand_n_words` (0.79), `sms_chars`–`n_options` (0.60), `sms_chars`–`sms_segments` (0.57), and `sms_segments`–`cand_n_words` (0.51) (Fig. 5). The correlation between `n_words_stem` and `sms_chars` on the full table was initially 0.92, while on the short MCQ/TF items it is 0.46. The drop in correlation was because the long reading passages that drove the earlier value are now out of scope.

VIF step. After capping and the learned log transform, the variance inflation factors were 10.31 for `sms_chars`, 6.37 for `cand_n_words`, and below 2.2 for the rest. Our protocol (VIF > 10) therefore removed `sms_chars`; the maximum VIF of the remaining features is 2.25. This agrees with Phase 1's expectation that one of the length counts would be removed. In the data, `sms_segments` is two-valued (151 items with one SMS part, 19 with two) and carries a different signal than the character count, while 90% of the variance of `sms_chars` is explained by the other length features ($1 - 1/10.31$). I noted that 10.31 is close to the threshold; a VIF of 9.7 before the selective log rule (Section VI) would have kept it. I kept the rule as written rather than tune the threshold to the result.

Association with the target. Mutual information against `is_correct` (Fig. 6) is highest for `cand_position` (0.067), `cand_len_ratio` (0.058), `cand_n_words` (0.053), `cand_is_longest` (0.036), and `item_type` (0.021). The item-level counts (`n_words_question`, `sms_chars`, `sms_segments`) and `class_level` have MI of 0, as Phase 1 predicted for features that are constant within an item. The correct candidate tends to be longer than its distractors (Spearman 0.29 with `cand_len_ratio`, 0.28 with `cand_is_longest`) and sits earlier in the list (−0.11 with position). These are surface cues, not subject knowledge. Because item-level MI is zero, I could not use MI to choose between collinear partners as Phase 1 intended, so VIF decided alone.

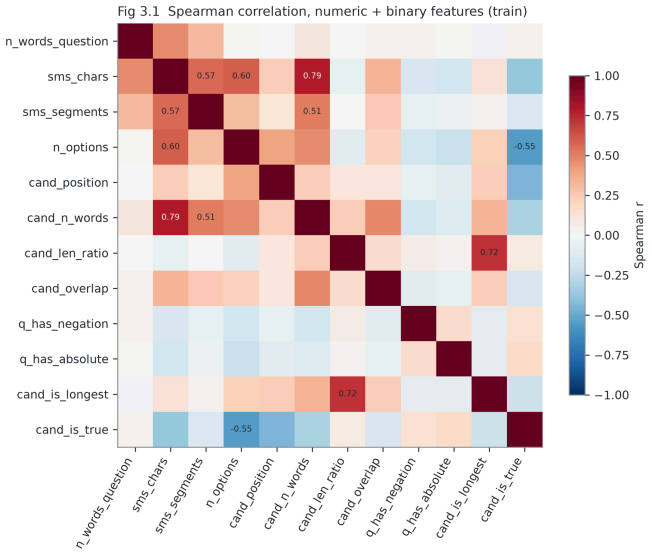


Fig. 4. Spearman correlation matrix for the numeric and binary features (training split). Cells with $|r| \geq 0.5$ are annotated.

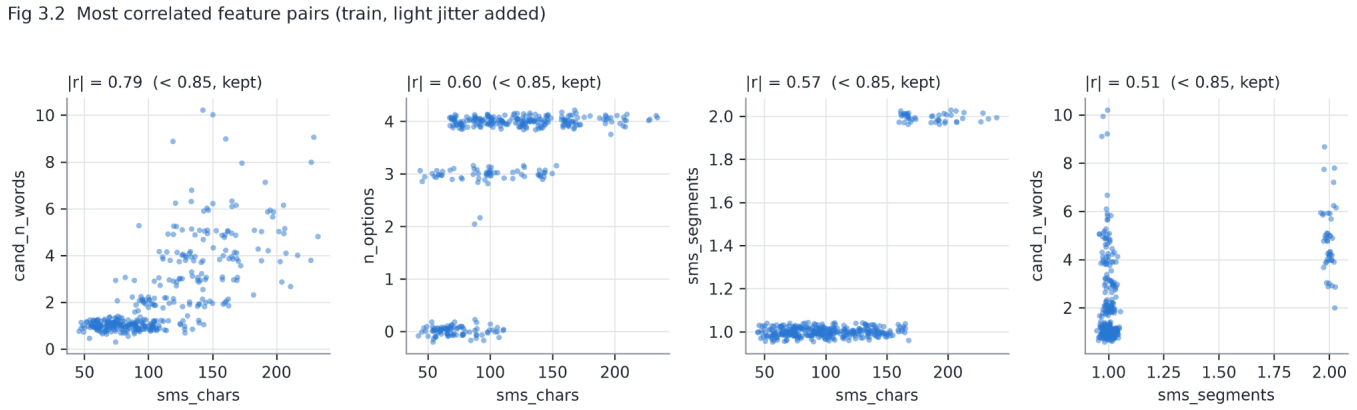


Fig. 5. The four most correlated feature pairs. None reaches the 0.85 threshold, so the VIF step decided which feature to drop.

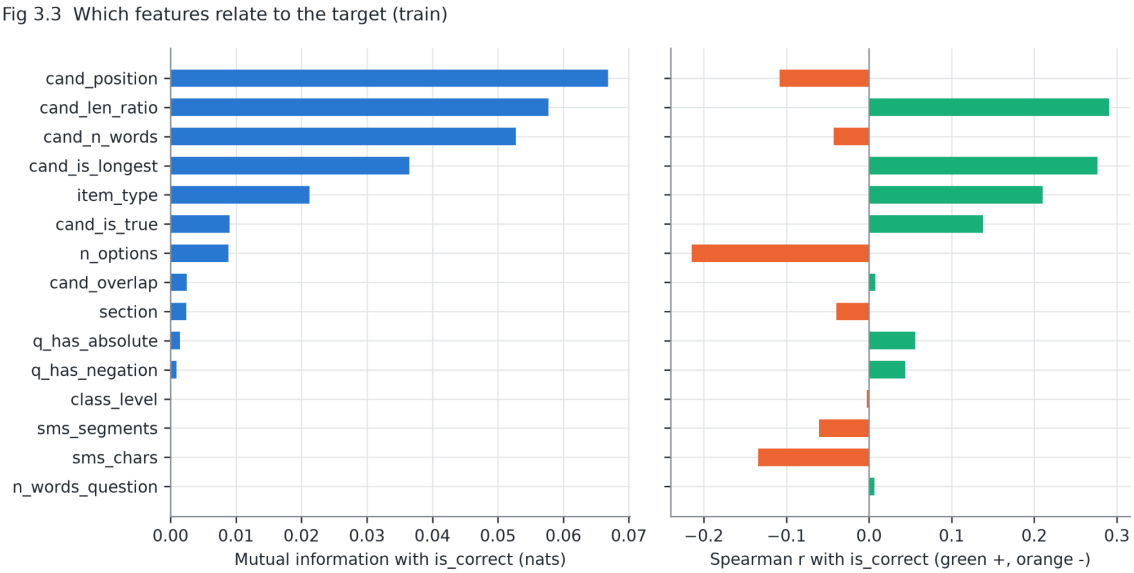


Fig. 6. Mutual information (left) and Spearman correlation (right) of each feature with `is_correct`, training split.

VI. TRANSFORMATIONS, ENCODING AND SCALING

Log transform. Phase 1 planned log(1 + x) for every count. Applying this made `n_words_question` worse (skewness +0.14 → −0.62), so the pipeline learns the rule from the training split: log(1 + x) is applied only to columns whose skewness after capping exceeds 1. That selects `sms_segments`, `cand_n_words` and `cand_overlap`. The log reduced `cand_n_words` from 1.05 to 0.73 but left `sms_segments` (2.32) and `cand_overlap` (2.22) almost unchanged, because they are essentially two-valued or zero-inflated; a transform cannot fix that, and tree models are unaffected by it (Fig. 7).

Scaling. Phase 1 considered MinMax scaling; the executed step is StandardScaler after the log step. On the training split, every scaled numeric column has mean ≈ 0 (largest absolute mean 1.2×10^{-15}) and standard deviation 1.000. On the test split, which uses training parameters, the largest absolute mean is 0.49 and the standard deviation ranges from 0.88 to 1.37, which is the expected sign that no test information entered the scaler.

Encoding. `class_level` (4 < 5 < 6) uses ordinal encoding; `item_type` (2 values) and `section` (4 values seen in the usable items) use one-hot encoding with `handle_unknown="ignore"`; four binary flags pass through. `subject`, `n_blanks`, `n_subitems`, `uses_unicode`, and the picture and computer flags were dropped from the model table because they are constant after scoping to usable MCQ/TF items.

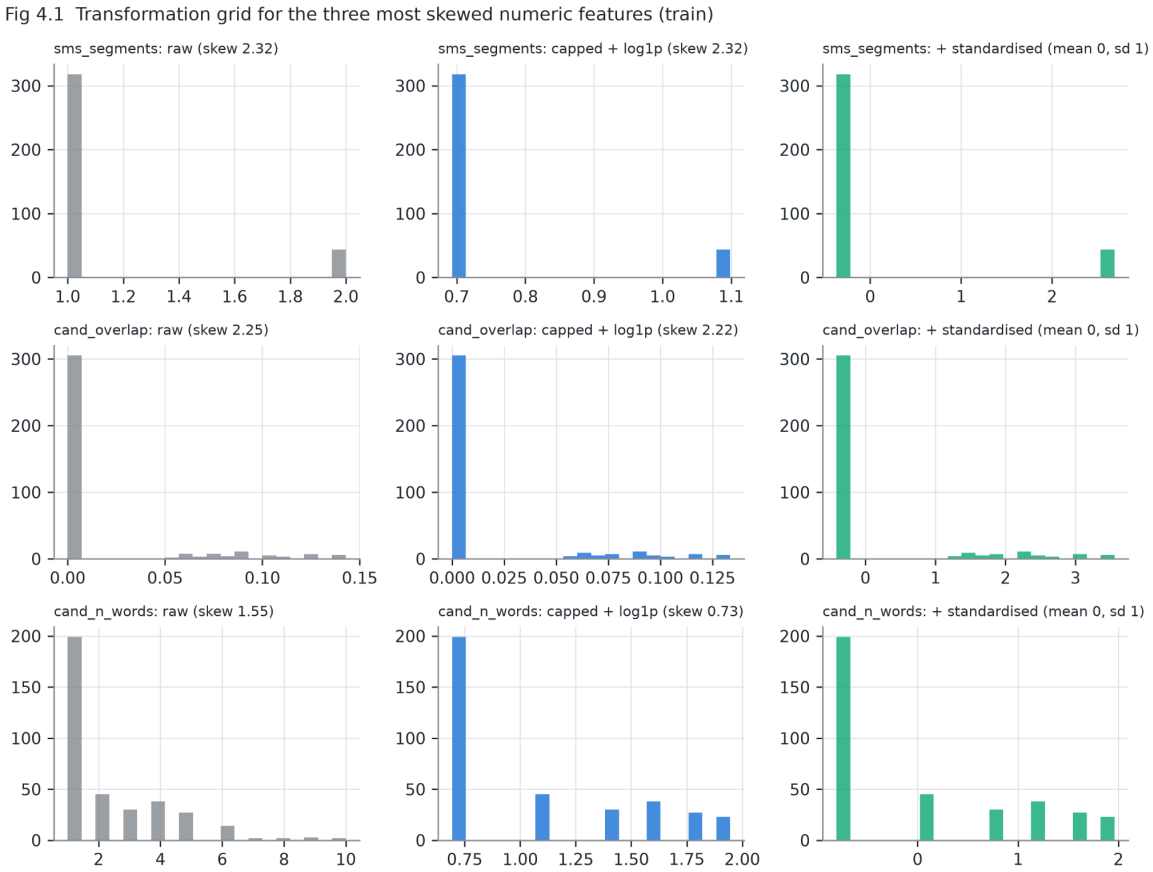


Fig. 7. Transformation stages for the three most skewed features that receive the log step: raw, capped plus log, and standardised (training split).

VII. DIMENSIONALITY REDUCTION

Principal component analysis was run on the 18-column scaled training matrix as a diagnostic (Fig. 8). The first component explains 28.2% of the variance and 9 components reach 95%. The variance spreads over many components, so the features do not collapse into a few directions. All 18 interpretable columns were kept for training, and PCA was not applied, in line with the plan to keep features explainable. This also leaves room to add reply-based features later.

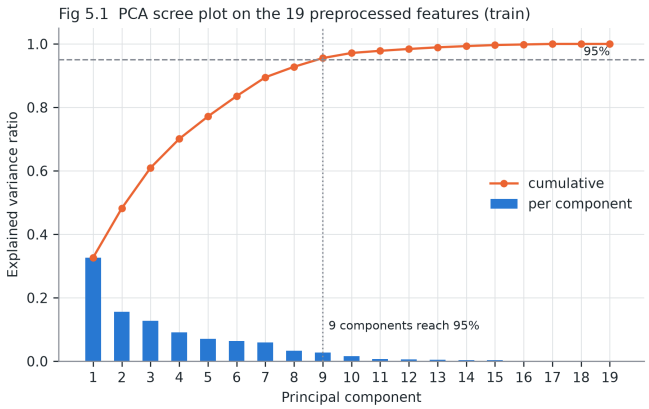


Fig. 8. PCA scree and cumulative variance on the preprocessed training matrix. Eight components reach 90%, and nine reach 95%.

VIII. LEAKAGE-FREE ARCHITECTURE

All learned steps (median and mode imputers, Winsorizer limits, log rule, VIF drop, scaler means, one-hot categories) sit inside one scikit-learn `ColumnTransformer` and `Pipeline` built in `src/pipeline_builder.py`, with custom transformers in `src/transformers.py`. It is fitted on the training split only, and validation and test sets are

transformed with the stored parameters, which avoids leaking statistics across splits [2].

The 520 rows come from 170 questions and are divided using 70/15/15 at the question level and stratified by item type: 118 questions (362 rows) for training, 26 (78 rows) for validation and 26 (80 rows) for test. A check confirms that no `group_id`, which is a hash of class and normalised question text, appears in two splits. Without this, a model would see the same question's options in training and test.

Phase 1 also planned 5-fold `StratifiedGroupKFold` inside the training split; that belongs to model tuning and has not been run yet. The real-life test set is still locked.

IX. PIPELINE OUTPUT VERIFICATION

Table III records the dimensions at each stage. No row was deleted by capping or imputation. Fifteen input features became 18 columns: 7 numeric (one was removed by the VIF step), 6 one-hot columns for the 2 nominal features, 1 ordinal, and 4 binary. Retention is therefore 14 of 15 input features (93%), with the dropped feature's variance covered by the others (VIF 10.31). The matrices contain no missing values.

TABLE III
MATRIX DIMENSIONS N×P AT EACH STAGE

Stage	N×P	Note
Raw extract	965×28	5 item types
Cleaned MCQ/TF	359×37	170 usable
Merged Q&A	170×20	one-to-one on ID
Candidate table	520×22	15 model features
X_train	362×18	after pipeline
X_val	78×18	train parameters
X_test	80×18	train parameters

X. STRATEGY VERSUS REALITY

Table IV compares each Phase 1 decision with what was executed.

TABLE IV
DECISION LOG: PHASE 1 PLAN AGAINST PHASE 2 EXECUTION

Step	Phase 1 plan	Phase 2 execution	Evidence (figure)
Scope	All five item types, 538 items, ≈2,152 rows	MCQ and TF only, 170 items, 520 rows	21 bad keys set to missing, 150 items with no key (Table I)
Target Y	Language-model replies, 1 correct : 3 wrong (25% positive)	Options and True/False as candidates, key as label (32.6% positive)	No replies exist yet (Fig. 2)
Missingness test	Chi-square and Cramér's V; Little's test set aside	Same; Little's test undefined (0 df, 2 patterns)	V = 0.703 with class, $p < 0.001$ (Fig. 1)
Missing keys	Never impute; keep row, flag, exclude from training	Same; 150 no-key and 10 untrusted rows excluded	48.2% of keys missing (Fig. 1)
Feature imputation	Median / mode in pipeline	Kept; fills 0 cells now	0 missing of 7,800 cells
Outlier rule	IQR, 99th percentile when IQR = 0; cap, never delete	Same, plus exemption for bounded counts	`n_options` fence would cap 24.3% (Fig. 3)
Outliers capped	Expected many (skew 1.8–13.5)	47 of 2,896 cells (1.6%)	Skew of `cand_len_ratio` 1.17 → 0.21 (Fig. 3)
Collinearity screen	Spearman $\|r\| \geq 0.85$, then VIF > 10	Same; no pair above 0.85, VIF dropped `sms_chars`	Max $\|r\| = 0.79$; VIF 10.31 → 2.25 (Figs. 4, 5)
Feature to drop	`sms_segments` (expected)	`sms_chars`	`sms_segments` VIF 2.14 (Fig. 5)
Tie-break by MI	MI between collinear partners	Not usable: MI = 0 for all item counts	MI 0.000 (Fig. 6)
Transformation	log(1 + x) on all counts	Learned: only if skew > 1 after capping	`n_words_question` +0.14 → −0.62 if logged (Fig. 7)
Scaling	StandardScaler (MinMax considered)	StandardScaler	Train mean ≈ 0, sd = 1.000 (Fig. 7)
Encoding	Ordinal class, one-hot nominal	Same; constant columns dropped	15 inputs → 18 columns
Split	70/15/15 by question group; 5-fold grouped CV	70/15/15 done; CV deferred to modelling	118 / 26 / 26 questions, no group in two splits
PCA	Diagnostic on embeddings	Diagnostic on 18 scaled columns	90% at 8, 95% at 9 components (Fig. 8)
Embedding features	Frozen sentence embeddings for similarity	Not yet built; token overlap only	`cand_overlap`, MI 0.002 (Fig. 6)

XI. LIMITATIONS AND NEXT STEPS

The target is derived from options, not from pupil replies, so results in this table measure option selection and will overstate performance on real SMS text. Candidate position and length are shortcut cues (Figs. 2 and 6); models must be reported with these features

removed. Answer keys come from Teacher's Guides, and a key shifted by one line would pass every automatic check, so a manual audit of a random sample is still needed. The next phase adds language-model replies, the frozen-embedding similarity features, grouped cross-validation and a real-life labelled test set.

XII. CONCLUSION

The Phase 1 pipeline extracted all questions from the books, but four decisions changed. Scope narrowed from 538 to 170 items, the target came from options, bounded counts were exempted from capping, and the log transform and the dropped feature were decided by training-split statistics. Each change is backed by a figure and a number, and the final matrices are leakage-free, complete, and fully regenerated by `python -m src.figures`.

REFERENCES

[1] Rwanda Education Board, Science and elementary technology, pupil's books and teacher's guides, primary 4–6. Kigali, Rwanda: Rwanda Education Board, 2026

[2] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," ACM Trans. Knowl. Discov. Data, vol. 6, no. 4, Art. no. 15, 2012.